C++ COMPETITIVE PROGRAMMING TRD

Beginner-Friendly Team Reference Document

For a 2–2.5 hour programming contest

Focus topics
C++ basics • Arrays • Vectors • Strings • Sorting • Searching • Frequency counting • Two pointers • Prefix sums • Stacks • Queues • Linked lists • Recursion • Basic maths • Complexity

Made for quick revision and understanding—not as a substitute for practising problems.

Important competition-rule note

This document is a study and reference guide. Your competition explicitly permits standard snippets such as fast I/O templates and basic macros, while restricting prohibited pre-written code. Confirm with the organizers whether complete implementations may be included in the TRD or copied during the contest. Follow their interpretation of the rules.

1. Basic C++ template

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <stack>
#include <queue>
#include <numeric>
using namespace std;

int main() {
ios::sync_with_stdio(false);
cin.tie(nullptr);

// Write your solution here

return 0;
}
```

Fast I/O makes input and output more efficient. Use only the parts permitted by your contest rules.

Input and output

```
int n;
cin >> n;
cout << n << '\\n';
```

• **int:** integer; **long long:** larger integer; **double:** decimal; **char:** one character; **string:** text; **bool:** true/false.

• **cin >> s** reads one word; **getline(cin, s)** reads a full line.

• If using getline after formatted input, handle the leftover newline with cin.ignore() when appropriate.

• Use == for comparison and = for assignment.

2. Conditions, loops and modulo

```
if (n > 0) {
cout << "Positive";
} else if (n < 0) {
cout << "Negative";
} else {
cout << "Zero";
}

for (int i = 0; i < n; i++) {
cout << i << " ";
}

if (n % 2 == 0)
cout << "Even";
else
cout << "Odd";
```

For n elements indexed from 0, valid indices are 0 through n - 1. The expression a % b gives the remainder, provided b is not zero.

3. Time complexity

Complexity	Meaning	Example
O(1)	Constant	Access one array element
O(log n)	Logarithmic	Binary search
O(n)	Linear	One pass through data
O(n log n)	Linearithmic	Efficient comparison sorting
$O(n^2)$	Quadratic	Two nested loops over n elements

• One loop over n elements is usually O(n).

• Consecutive loops: add their costs and keep the dominant term.

• Repeatedly halving a search range is O(log n).

• Choose an approach based on input constraints and the time limit.

4. Arrays

Find maximum, minimum and sum

```
// Assumes n > 0
int mx = arr[0];
int mn = arr[0];
long long sum = 0;

for (int i = 0; i < n; i++) {
if (arr[i] > mx) mx = arr[i];
if (arr[i] < mn) mn = arr[i];
sum += arr[i];
}
```

Linear search

```
int pos = -1;

for (int i = 0; i < n; i++) {
if (arr[i] == target) {
pos = i;
break;
}
}
// pos == -1 means not found
```

Reverse an array

```
int left = 0, right = n - 1;

while (left < right) {
swap(arr[left], arr[right]);
left++;
right--;
}
```

Count occurrences

```
int count = 0;
for (int i = 0; i < n; i++) {
if (arr[i] == target) count++;
}
```

Array indexing and element access are O(1). Searching an unsorted array takes O(n). Always handle n == 0 before accessing arr[0].

5. Vectors

```
vector<int> v(n); // n elements
v.push_back(x); // add at end
v.pop_back(); // remove last element
v.size(); // number of elements
v.empty(); // true if empty
v[i]; // element at index i
v.front(); // first element
v.back(); // last element
```

Check that a vector is not empty before using front(), back() or pop_back().

```
int n;
cin >> n;
vector<int> v(n);

for (int i = 0; i < n; i++) cin >> v[i];

for (int i = 0; i < n; i++) cout << v[i] << " ";
```

6. Strings

```
string s = "hello";
s.size(); // length
s[0]; // first character
reverse(s.begin(), s.end());
```

Palindrome check

```
bool palindrome = true;
int left = 0;
int right = (int)s.size() - 1;

while (left < right) {
if (s[left] != s[right]) {
palindrome = false;
break;
}
left++;
right--;
}
```

A palindrome reads the same forward and backward, such as madam. Handle an empty string safely.

Count a character

```
int count = 0;
for (int i = 0; i < (int)s.size(); i++) {
if (s[i] == target) count++;
}
```

7. Sorting

```
sort(v.begin(), v.end()); // ascending
sort(arr, arr + n); // plain array ascending
reverse(v.begin(), v.end());
```

For descending order, a common form is sort(v.begin(), v.end(), greater()); include when needed. Sorting with sort() typically takes O(n log n). Sorting changes the original order.

Binary search requires sorted data.

8. Searching

Linear search

Check each element one by one. Works on sorted or unsorted data. Time complexity: O(n).

Binary search

• Use only on sorted data.

• Set low = 0 and high = n - 1.

• Calculate mid = low + (high - low) / 2.

• If arr[mid] equals the target, it is found.

• If the target is smaller, set high = mid - 1; otherwise set low = mid + 1.

• Continue while low <= high. Time complexity: O(log n).

```
if (binary_search(v.begin(), v.end(), target)) {
cout << "Found";
} else {
cout << "Not found";
}
```

binary_search() requires and sorted data.

9. Frequency counting

Frequency means how many times a value appears. For array [2, 3, 2, 5, 3, 2], frequency of 2 is 3.

```
// Only when values are from 0 to 100
int freq[101] = {0};

for (int i = 0; i < n; i++) {
freq[arr[i]]++;
}
// freq[x] is the count of x
```

This array-based approach is valid only when values fit the frequency array's index range.

10. Two pointers

Two pointers are two indices that move through data. A common use is finding a pair in a sorted array that adds to a target.

• Start left at 0 and right at n - 1.

• If arr[left] + arr[right] equals target, the pair is found.

• If the sum is too small, increment left.

• If the sum is too large, decrement right.

• Stop when left >= right.

On sorted input this pattern can run in O(n). The pointer movement rule does not generally work on unsorted input.

11. Prefix sums

Prefix sums store cumulative totals. For [2, 4, 1, 3], prefix sums are [2, 6, 7, 10].

```
prefix[0] = arr[0];

for (int i = 1; i < n; i++) {
prefix[i] = prefix[i - 1] + arr[i];
}
```

• Sum from index 0 to r: prefix[r].

• Sum from l to r when l > 0: prefix[r] - prefix[l - 1].

• When l == 0, the sum is prefix[r].

• Building prefix sums takes O(n); each range query takes O(1).

• Use long long if sums may be large.

12. Stacks

A stack follows LIFO: Last In, First Out. The last item added is removed first.

```
stack<int> st;
st.push(10);
st.push(20);

st.top(); // inspect top: 20
st.pop(); // remove top
```

```
st.empty(); // check whether empty
st.size(); // number of elements
```

pop() removes an element but does not return it. Check empty() before top() or pop(). Common uses include valid parentheses, reversing items and undo operations.

13. Queues

A queue follows FIFO: First In, First Out. The first item added is removed first.

```
queue<int> q;
q.push(10);
q.push(20);

q.front(); // first item: 10
q.pop(); // remove first item
q.empty(); // check whether empty
q.size(); // number of elements
```

Check empty() before front() or pop().

14. Linked lists

A linked list consists of nodes. Each node stores data and a pointer to the next node.

```
struct Node {
int data;
Node* next;
};
```

• head points to the first node.

• head == nullptr means the list is empty.

• The last node points to nullptr.

• Traversal starts at head and follows next until nullptr.

• Searching or accessing by position generally takes O(n).

• Inserting or deleting at the head takes O(1), assuming the list is already available.

Be careful with null pointers, link updates and memory allocated dynamically.

15. Functions and recursion

Function

```
int add(int a, int b) {
return a + b;
}
// Example: add(3, 4) returns 7
```

Recursion

A recursive function calls itself. Every recursive solution needs a base case to stop and a recursive case that moves toward the base case.

Factorial: 0! = 1 and n! = n × (n - 1)! for n > 0. For example, 4! = 24. Watch for missing base cases, excessive recursion depth and overflow.

16. Basic mathematics

GCD

GCD is the greatest positive integer that divides both numbers. Use gcd(a, b) from in C++17 or later.

```
int g = gcd(a, b);
```

LCM

For positive integers: LCM(a, b) = (a / GCD(a, b)) * b. Dividing first can reduce overflow risk, but the result may still be too large.

Prime numbers

• A prime is an integer greater than 1 with exactly two positive divisors.

• Handle n < 2 as not prime.

• It is enough to check possible divisors up to the square root of n.

• A condition such as i <= n / i avoids calculating i * i, which can overflow.

Useful formulas

• Even: n % 2 == 0

• Odd: n % 2 != 0

• Sum of first n positive integers: n * (n + 1) / 2

• Modulo a % b gives the remainder when b != 0.

17. Common mistakes

• Off-by-one errors: for n elements, valid indices are 0 through n - 1.

• Empty input: never access arr[0] when n == 0.

• Integer overflow: use long long for potentially large totals or products.

• Binary search on unsorted data.

• Missing return values in non-void functions.

• Infinite loops caused by boundaries that do not change.

• Using stack top() or queue front() on an empty container.

• Printing prompts or extra text not requested by the problem.

• Uninitialized counters, sums or flags.

18. Contest strategy

• Read the whole statement and identify input, output and constraints.

• Work through the sample manually before coding.

• Start with problems whose logic you understand.

• All problems have equal points, so secure easier accepted solutions first.

• If stuck without a promising approach, switch problems and return later.

• Test minimum inputs, repeated values, boundary values and overflow cases.

• Keep track of wrong submissions because each incurs a penalty.

• If in a team, communicate your approach and share the computer efficiently using only permitted methods.

19. Advanced topics to learn later

Trees, graphs, dynamic programming, advanced data structures and advanced number theory are deliberately not required in this beginner guide. Learn them later if your syllabus or contest problem set calls for them.

Final rules reminder

This document is for study and reference. Your contest explicitly allows standard fast-I/O snippets and basic macros, while restricting prohibited pre-written code. Ask organizers whether complete implementations may be included in the TRD or copied during the contest. Follow the official contest rules.